

LAPORAN PROYEK BIG DATA
Pengembangan Sistem Big Data Terintegrasi untuk Analisis
Kecelakaan Jalan Raya di Inggris Menggunakan Apache Spark dan
SQL–NoSQL



Di Susun Oleh :

11423002	Hizkia Abednego Sipayung
11423044	Nathanael T.J. Tampubolon
11423045	Whisnu Abraham Saragih
11423055	Leoni Nazwa Friskilla Sibuea
11423057	Imel Sari Sinaga

PROGRAM STUDI SARJANA TERAPAN TEKNOLOGI REKAYASA PERANGKAT
LUNAK

FAKULTAS VOKASI

INSTITUT TEKNOLOGI DEL

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kecelakaan jalan raya merupakan salah satu masalah penting dalam sistem transportasi karena dapat menimbulkan korban jiwa, kerugian ekonomi, dan gangguan terhadap mobilitas masyarakat. Secara global, WHO mencatat bahwa kematian akibat kecelakaan lalu lintas masih mencapai sekitar 1,19 juta jiwa per tahun [1]. Di Inggris, laporan Reported Road Casualties Great Britain 2023 mencatat 132.977 korban kecelakaan pada tahun 2023, termasuk 29.711 korban dalam kategori *killed or seriously injured* [2]. Data tersebut menunjukkan bahwa kecelakaan masih menjadi persoalan yang perlu dianalisis secara lebih serius untuk mengetahui pola kejadian, faktor risiko, dan wilayah yang rawan kecelakaan.

Data kecelakaan jalan raya memiliki karakteristik yang cukup kompleks karena memuat berbagai informasi, seperti waktu kejadian, lokasi, kondisi jalan, cuaca, jenis kendaraan, jumlah korban, dan tingkat keparahan kecelakaan. Jumlah data yang besar membuat proses pengolahan tidak cukup dilakukan secara manual atau dengan cara sederhana. Data perlu melalui proses pembersihan, pengolahan, penyimpanan, analisis, dan visualisasi agar dapat menghasilkan informasi yang mudah dipahami dan bermanfaat.

Berdasarkan permasalahan tersebut, proyek ini mengembangkan sistem Big Data terintegrasi untuk analisis kecelakaan jalan raya di Inggris. Apache Spark digunakan untuk memproses data dalam jumlah besar, mulai dari preprocessing, agregasi, analisis pola, hingga clustering hotspot kecelakaan. Spark juga mendukung analisis data skala besar dan machine learning melalui MLlib, sehingga sesuai untuk mengolah dataset kecelakaan yang memiliki banyak variabel.

Selain pemrosesan data, proyek ini juga menggunakan PostgreSQL dan MongoDB sebagai media penyimpanan. PostgreSQL digunakan untuk menyimpan data terstruktur dan hasil agregasi, sedangkan MongoDB digunakan untuk menyimpan data dalam bentuk dokumen yang lebih fleksibel. MongoDB cocok digunakan ketika data memiliki struktur yang dapat berkembang atau disusun dalam bentuk dokumen, misalnya data lokasi, waktu, kondisi jalan, dan kendaraan.

Melalui sistem ini, data kecelakaan yang awalnya berupa data mentah dapat diolah menjadi informasi yang lebih jelas melalui dashboard interaktif. Analisis yang dilakukan mencakup distribusi tingkat keparahan kecelakaan, tren kecelakaan berdasarkan waktu, perbandingan weekday dan weekend, faktor jalan dan cuaca, serta identifikasi wilayah rawan kecelakaan. Dengan demikian, proyek ini diharapkan dapat membantu memahami pola kecelakaan jalan raya di Inggris dan mendukung pengambilan keputusan berbasis data dalam bidang keselamatan transportasi.

1.2 Rumusan Masalah

Rumusan masalah dalam proyek ini berikut ini :

1. Bagaimana membangun sistem Big Data terintegrasi menggunakan Apache Spark, PostgreSQL, dan MongoDB untuk mengolah data kecelakaan jalan raya di Inggris dalam jumlah besar?
2. Bagaimana melakukan analisis data kecelakaan jalan raya berdasarkan tingkat keparahan, waktu kejadian, kondisi jalan, cuaca, jenis kendaraan, dan lokasi untuk mengetahui pola serta faktor risiko kecelakaan?
3. Bagaimana menyajikan hasil analisis data kecelakaan jalan raya ke dalam dashboard interaktif yang informatif dan mudah dipahami?

1.3 Tujuan Proyek

Tujuan dari proyek ini adalah sebagai berikut :

1. Membangun sistem Big Data terintegrasi menggunakan Apache Spark, PostgreSQL, dan MongoDB untuk mengolah data kecelakaan jalan raya di Inggris dalam jumlah besar.
2. Melakukan analisis data kecelakaan jalan raya untuk mengetahui pola kejadian, tingkat keparahan, faktor risiko, dan wilayah rawan kecelakaan berdasarkan waktu, kondisi jalan, cuaca, jenis kendaraan, dan lokasi.
3. Menyajikan hasil analisis data kecelakaan jalan raya dalam bentuk dashboard interaktif yang informatif dan mudah dipahami untuk mendukung analisis keselamatan transportasi berbasis data.

1.4 Manfaat Proyek

Manfaat yang diharapkan dari proyek ini adalah sebagai berikut :

1. Proyek ini membantu pengguna memahami pola kecelakaan jalan raya di Inggris, seperti kapan kecelakaan sering terjadi, faktor apa yang berpengaruh, dan wilayah mana yang lebih rawan.
2. Proyek ini menjadi penerapan sistem Big Data dengan Apache Spark, PostgreSQL, dan MongoDB untuk mengolah data kecelakaan dalam jumlah besar secara lebih terstruktur.
3. Proyek ini menyajikan hasil analisis dalam bentuk dashboard interaktif agar informasi tentang tren kecelakaan, tingkat keparahan, dan hotspot kecelakaan lebih mudah dibaca dan dipahami.

1.5 Pembagian Taks Anggota

Nama Anggota	Taks
Hizkia Abednego Sipayung	Dashboard Visualisasi, jurnal
Nathanael T.J. Tampubolon	Menghubungkan Neon PostgreSQL dan MongoDB Atlas, jurnal
Whisnu Abraham Saragih	pengujian pipeline, jurnal
Leoni Nazwa Friskilla Sibuea	Profiling, Apache Spark, preprocessing, Analisis data dan clustering hotspot, dokumen, jurnal
Imel Sari Sinaga	Data Ingestion dan Arsitektur Sistem, jurnal

BAB II

TINJAUAN PUSTAKA

2.1 Karakteristik Big Data

Big Data merupakan istilah yang digunakan untuk menggambarkan kumpulan data dengan ukuran besar, kompleks, dan terus bertambah sehingga sulit diolah menggunakan sistem pengolahan data tradisional. Big Data tidak hanya berkaitan dengan jumlah data, tetapi juga mencakup kecepatan data dihasilkan, keberagaman bentuk data, kualitas data, serta nilai informasi yang dapat diperoleh dari data tersebut. Karakteristik Big Data umumnya dikenal dengan konsep 5V, yaitu Volume, Velocity, Variety, Veracity, dan Value [3].

2.1.1 Volume

Volume adalah karakteristik Big Data yang menunjukkan besarnya jumlah data yang dihasilkan, disimpan, dan diproses dalam suatu sistem [3]. Semakin besar jumlah data yang digunakan, semakin besar pula kebutuhan sistem untuk menyimpan dan mengolah data tersebut. Pada proyek ini, karakteristik volume terlihat dari dataset kecelakaan jalan raya di Inggris yang memiliki lebih dari 300.000 baris data.

2.1.2 Velocity

Velocity adalah karakteristik Big Data yang berkaitan dengan kecepatan data dihasilkan, dikirim, dan diproses agar dapat segera digunakan untuk analisis [3]. Dalam proyek ini, velocity terlihat dari kebutuhan sistem untuk memproses data secara cepat melalui Apache Spark, terutama pada tahap cleaning, agregasi, dan analisis data kecelakaan.

2.1.3 Variety

Variety adalah karakteristik Big Data yang menunjukkan keberagaman bentuk dan jenis data, baik data terstruktur, semi-terstruktur, maupun tidak terstruktur [4]. Pada proyek ini, dataset kecelakaan memiliki berbagai jenis data, seperti data numerik, kategorikal, spasial, dan temporal. Contohnya adalah jumlah korban, jenis kendaraan, koordinat lokasi, kondisi cuaca, waktu kejadian, dan tingkat keparahan kecelakaan.

2.1.4 Veracity

Veracity adalah karakteristik Big Data yang berkaitan dengan kualitas, keakuratan, dan tingkat kepercayaan terhadap data yang digunakan dalam proses analisis [5]. Data yang tidak lengkap, tidak konsisten, atau memiliki kesalahan dapat memengaruhi hasil analisis. Oleh karena itu, pada proyek ini dilakukan proses preprocessing, seperti penanganan missing values, normalisasi kategori, validasi koordinat, dan penyesuaian tipe data.

2.1.5 value

Value adalah karakteristik Big Data yang menunjukkan nilai atau manfaat yang dapat dihasilkan dari data setelah melalui proses pengolahan dan analisis [3]. Pada proyek ini, nilai dari data

diperoleh melalui informasi tentang pola kecelakaan, tren waktu kejadian, faktor risiko, dan wilayah rawan kecelakaan yang ditampilkan melalui dashboard interaktif.

2.2 Teknologi yang Digunakan

2.2.1 Apache Spark

Apache Spark merupakan platform open-source yang digunakan untuk pemrosesan data skala besar dan cocok untuk pekerjaan analisis data serta machine learning [6]. Spark mendukung proses komputasi terdistribusi sehingga data dapat diproses secara paralel dan lebih efisien. Spark juga memiliki konsep Resilient Distributed Dataset atau RDD yang mendukung pemrosesan data dalam memori secara toleran terhadap kegagalan [6]. Dalam proyek ini, Apache Spark digunakan untuk membaca dataset kecelakaan, melakukan preprocessing, agregasi, analisis tren, dan clustering hotspot kecelakaan. Penggunaan Spark dipilih karena dataset yang digunakan memiliki jumlah data besar dan membutuhkan proses pengolahan yang efisien.

2.2.2 Spark MLlib

Spark MLlib adalah pustaka machine learning dari Apache Spark yang dirancang agar proses pembelajaran mesin dapat dilakukan secara lebih mudah dan skalabel [7]. Pada proyek ini, Spark MLlib digunakan untuk melakukan clustering menggunakan algoritma K-Means. Clustering ini bertujuan untuk mengelompokkan lokasi kecelakaan berdasarkan karakteristik tertentu, seperti koordinat lokasi, batas kecepatan, jumlah kendaraan, dan jumlah korban.

2.2.3 PostgreSQL

PostgreSQL merupakan sistem basis data relasional open-source yang menggunakan dan mengembangkan bahasa SQL untuk menyimpan serta mengelola data [8]. Dalam proyek ini, PostgreSQL digunakan untuk menyimpan data yang sudah dibersihkan dan hasil agregasi analisis. PostgreSQL cocok digunakan untuk data terstruktur, seperti jumlah kecelakaan berdasarkan tingkat keparahan, distrik, jenis kendaraan, dan waktu kejadian.

2.2.4 MongoDB

MongoDB merupakan database NoSQL berbasis dokumen yang mendukung model data fleksibel dan dapat digunakan untuk menyimpan data dalam bentuk dokumen [9]. Pada proyek ini, MongoDB digunakan untuk menyimpan data kecelakaan dalam struktur dokumen yang lebih fleksibel, seperti informasi lokasi, waktu, kendaraan, kondisi jalan, dan kualitas data. Penggunaan MongoDB membantu sistem dalam mengelola data yang tidak selalu harus disimpan dalam bentuk tabel relasional.

2.2.5 Streamlit

Streamlit adalah framework open-source berbasis Python yang digunakan untuk membuat aplikasi dan dashboard data secara interaktif [10]. Dalam proyek ini, Streamlit digunakan untuk menampilkan hasil analisis kecelakaan dalam bentuk grafik, tabel, dan visualisasi interaktif. Dashboard ini membantu pengguna melihat tren kecelakaan, distribusi tingkat keparahan, dan hotspot kecelakaan dengan lebih mudah.

2.2.6 Google Colab

Google Colab merupakan platform berbasis cloud yang dapat digunakan untuk menulis dan menjalankan kode Python melalui notebook. Dalam proyek ini, Google Colab digunakan sebagai lingkungan kerja untuk menjalankan kode Python dan Spark tanpa harus melakukan konfigurasi berat pada perangkat lokal. Platform ini juga memudahkan proses pengembangan karena dapat terhubung dengan Google Drive sebagai tempat penyimpanan dataset dan hasil analisis.

Bab III

METODOLOGI PENGEMBANGAN SISTEM

3.1 Pendekatan Pengembangan Sistem

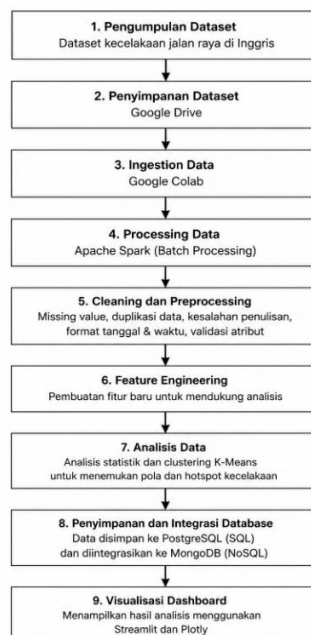
Proyek ini menggunakan pendekatan pipeline Big Data end-to-end. Pendekatan ini dipilih karena sistem tidak hanya membaca data, tetapi juga mengolah data dari tahap ingestion, cleaning, preprocessing, analisis, penyimpanan, sampai visualisasi. Pengolahan data dilakukan secara batch processing karena data yang digunakan berjumlah besar dan diproses dalam satu rangkaian tahapan. Dengan pendekatan ini, data mentah dapat diolah menjadi informasi yang lebih terstruktur dan mudah dianalisis melalui dashboard.

3.2 Tahapan Metodologi Pengembangan

Tahapan pengembangan sistem dimulai dari pengumpulan dataset kecelakaan jalan raya di Inggris, kemudian dataset disimpan di Google Drive dan dimuat ke Google Colab. Setelah itu, data diproses menggunakan Apache Spark untuk melakukan ingestion, cleaning, preprocessing, dan pembuatan fitur baru.

Data yang sudah bersih kemudian dianalisis untuk melihat pola kecelakaan berdasarkan waktu, wilayah, tingkat keparahan, kondisi jalan, cuaca, dan jenis kendaraan. Selain itu, digunakan K-Means untuk melihat hotspot kecelakaan berdasarkan lokasi.

Hasil pengolahan data disimpan ke PostgreSQL dan MongoDB. Integrasi dilakukan dengan memindahkan hasil query dari PostgreSQL ke MongoDB, lalu data divisualisasikan melalui dashboard menggunakan Streamlit dan Plotly.



Gambar 1. Tahapan Metodologi Pengembangan

3.3 Sumber dan Deskripsi Dataset

3.3.1 Informasi Dataset

Data yang digunakan dalam pengembangan sistem pipeline Big Data ini bersumber dari open dataset platform Kaggle dengan judul Road Accidents Dataset. Dataset ini berisi data kecelakaan jalan raya di Inggris dan dapat diakses secara publik melalui Kaggle. Dataset yang digunakan dalam proyek ini memiliki nama file Road Accident Data.csv.

Dataset ini dipilih karena memiliki jumlah data yang besar, yaitu 307.973 baris dan 21 kolom. Jumlah tersebut sudah sesuai dengan kebutuhan proyek Big Data karena melebihi batas minimal 100.000 record. Selain itu, dataset ini memiliki variasi data yang cukup beragam, seperti waktu kejadian, lokasi kecelakaan, tingkat keparahan, jumlah korban, jumlah kendaraan, kondisi cuaca, kondisi jalan, jenis jalan, batas kecepatan, dan jenis kendaraan. Variasi atribut tersebut membuat dataset ini sesuai untuk digunakan dalam analisis pola kecelakaan, clustering hotspot, integrasi database, dan visualisasi dashboard. Dataset ini dapat diakses secara publik melalui tautan resmi: <https://www.kaggle.com/datasets/atharvasoundankar/road-accidents-dataset>

3.3.2 Karakteristik dan Atribut Dataset

Dataset ini memiliki atribut yang menggambarkan kondisi kecelakaan lalu lintas secara cukup lengkap. Atribut tersebut mencakup waktu kejadian, lokasi kecelakaan, tingkat keparahan, jumlah korban, jumlah kendaraan, kondisi cuaca, kondisi jalan, jenis jalan, batas kecepatan, serta jenis kendaraan yang terlibat.

Dalam proyek ini, atribut waktu digunakan untuk melihat pola kecelakaan berdasarkan jam dan hari. Atribut lokasi digunakan untuk menganalisis wilayah rawan kecelakaan, sedangkan latitude dan longitude digunakan untuk mendukung analisis hotspot menggunakan K-Means. Atribut lain seperti kondisi cuaca, kondisi jalan, dan jenis kendaraan digunakan untuk memperkuat hasil analisis pada dashboard.

3.4 Arsitektur Sistem

Arsitektur sistem pada proyek ini terdiri dari beberapa komponen utama, yaitu Google Drive, Google Colab, Apache Spark, PostgreSQL, MongoDB, serta dashboard Streamlit dan Plotly. Setiap komponen memiliki peran yang saling terhubung dalam proses pengolahan data. Google Drive digunakan sebagai tempat penyimpanan dataset, sedangkan Google Colab digunakan sebagai lingkungan kerja untuk menjalankan kode program. Apache Spark berperan dalam memproses data, PostgreSQL digunakan untuk menyimpan data hasil pengolahan secara relasional, dan MongoDB digunakan untuk menyimpan hasil integrasi data dalam bentuk dokumen. Data yang tersimpan kemudian digunakan sebagai sumber visualisasi pada dashboard.

3.5 Metode Cleaning dan Preprocessing Data

Cleaning dan preprocessing dilakukan untuk membuat data lebih rapi dan siap dianalisis. Tahap ini meliputi pemeriksaan missing value, data duplikat, kesalahan penulisan, serta penyesuaian

format tanggal dan waktu. Setelah itu, nama kolom disesuaikan agar lebih mudah digunakan dalam proses pengolahan. Sistem juga membuat beberapa fitur baru, seperti penanda akhir pekan, tingkat keparahan kecelakaan, dan indikator kecelakaan berat. Data yang sudah bersih kemudian digunakan untuk analisis.



Gambar 2. Tahapan Cleaning dan Preprocessing Data

3.6 Rancangan Integrasi SQL dan NoSQL

Integrasi SQL dan NoSQL pada proyek ini dilakukan dengan menghubungkan PostgreSQL dan MongoDB. PostgreSQL digunakan sebagai database relasional untuk menyimpan data hasil pengolahan, sedangkan MongoDB digunakan untuk menyimpan hasil ringkasan data dalam bentuk dokumen.

Pada proyek ini, PostgreSQL dijalankan melalui layanan Neon PostgreSQL. Data yang sudah dibersihkan dan dianalisis dari Google Colab kemudian disimpan ke dalam beberapa tabel di PostgreSQL. Setelah itu, hasil query tertentu dari PostgreSQL dipindahkan ke MongoDB agar dapat digunakan kembali untuk kebutuhan visualisasi dashboard.

3.6.1 Rancangan PostgreSQL

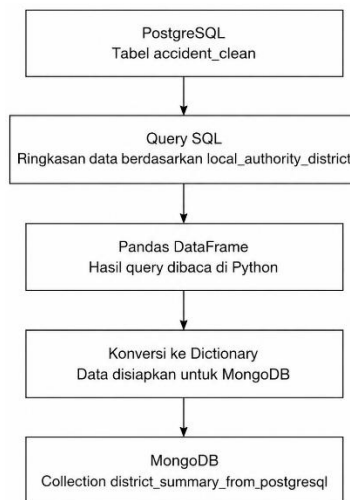
PostgreSQL digunakan sebagai database relasional untuk menyimpan data yang sudah melalui proses cleaning, preprocessing, dan analisis. Pada proyek ini, PostgreSQL dijalankan melalui layanan Neon dan dihubungkan dengan Google Colab menggunakan SQLAlchemy serta psycopg2. Data utama disimpan dalam tabel `accident_clean`, sedangkan hasil analisis disimpan dalam beberapa tabel ringkasan seperti `district_summary`, `hourly_trend`, `monthly_trend`, `severity_summary`, dan `speed_limit_summary` agar lebih mudah dikelola dan dianalisis menggunakan query SQL.

3.6.2 Rancangan MongoDB

MongoDB digunakan sebagai database NoSQL untuk menyimpan hasil ringkasan data yang berasal dari PostgreSQL dalam bentuk dokumen. Pada proyek ini, data hasil query dari PostgreSQL dimasukkan ke collection `district_summary_from_postgresql` yang berisi informasi ringkasan kecelakaan berdasarkan district, seperti total kecelakaan, total korban, dan tingkat kecelakaan berat. Data ini kemudian digunakan sebagai sumber untuk visualisasi dashboard.

3.6.3 Alur Integrasi SQL dan NoSQL

Alur integrasi dimulai dari data bersih yang sudah tersimpan di PostgreSQL. Sistem menjalankan query untuk mengambil ringkasan kecelakaan berdasarkan `local_authority_district`, kemudian hasil query tersebut dibaca menggunakan `pandas`. Setelah itu, data diubah ke dalam format dictionary dan dimasukkan ke MongoDB melalui collection `district_summary_from_postgresql`. Dengan alur ini, data dari PostgreSQL dapat dipindahkan ke MongoDB dan digunakan kembali sebagai sumber data dashboard.



Gambar 3. Alur Integrasi SQL dan NoSQL

3.7 Rancangan Visualisasi Dashboard

Dashboard dirancang untuk menampilkan hasil analisis data kecelakaan dalam bentuk yang lebih mudah dipahami. Visualisasi dibuat menggunakan Streamlit dan Plotly, dengan sumber data dari hasil pengolahan yang tersimpan di database. Informasi yang ditampilkan pada dashboard meliputi jumlah kecelakaan, jumlah korban, tingkat keparahan, wilayah dengan kecelakaan tertinggi, serta ringkasan data berdasarkan district. Dashboard juga dirancang agar pengguna dapat melihat pola kecelakaan melalui grafik, tabel, dan insight singkat.

3.8 Metode Pengujian Sistem

Pengujian sistem dilakukan dengan memastikan setiap bagian pipeline berjalan tanpa error, mulai dari pembacaan dataset, proses cleaning dan preprocessing, penyimpanan data ke PostgreSQL, integrasi data ke MongoDB, sampai visualisasi dashboard menggunakan Streamlit dan Plotly.

BAB IV

IMPLEMENTASI

4.1 Ingestion Dataset

Tahap ingestion dataset dilakukan dengan memuat data kecelakaan jalan raya dari Google Drive ke Google Colab. Dataset yang digunakan adalah Road Accident Data.csv. Dataset tersebut disimpan di Google Drive agar mudah diakses melalui notebook, kemudian dibaca untuk diproses pada tahap berikutnya. Tahap ini menjadi langkah awal dalam pipeline karena seluruh proses pengolahan data bergantung pada keberhasilan pembacaan dataset.

```
[11] from google.colab import drive
    drive.mount('/content/drive')

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[12] !ls "/content/drive/MyDrive"
    'colab Notebooks'  PA3KEL10

[13] !ls "/content/drive/MyDrive/PA3KEL10/proyek_bidat/data/raw"
    'Road Accident Data.csv'

[14] !ls "/content/drive/MyDrive/PA3KEL10/proyek_bidat"
    dashboard docs  README.md  slides  untitled0.ipynb
    data      notebooks  requirements_colab.txt  src      video
```

Gambar 4. Proses Akses Dataset dari Google Drive

4.2 Dataset Profilling

Dataset profiling dilakukan untuk melihat kondisi awal data sebelum masuk ke tahap cleaning dan preprocessing. Hasil profiling menunjukkan bahwa dataset memiliki 307.973 baris dan 21 kolom, Missing value paling banyak terdapat pada kolom Carriageway_Hazards, sedangkan kolom Time hanya memiliki sedikit data kosong. Dataset ini mencakup data kecelakaan tahun 2021 dan 2022, dengan kategori kecelakaan terbanyak yaitu Slight, kemudian Serious, dan Fatal. Pada tahap ini, sistem juga menambahkan beberapa kolom bantu, yaitu Accident_Severity_Clean, date_parsed, dan hour untuk mendukung proses analisis awal. Selain itu, hasil profiling menunjukkan bahwa beberapa district dengan jumlah kecelakaan tinggi adalah Birmingham, Leeds, Manchester, Bradford, dan Westminster. Jenis kendaraan yang paling banyak terlibat adalah Car, disusul Van/Goods, Motorcycle, dan Bus or coach.

```
Dataset Profiling

import pandas as pd
import json

df_pd = pd.read_csv(RAW_DATA_PATH)

original_rows = df_pd.shape[0]
original_columns = df_pd.shape[1]
original_column_names = df_pd.columns.tolist()

# Buat salinan khusus untuk profiling agar data asli tidak berubah
df_profile = df_pd.copy()

df_profile['Accident_Severity_Clean'] = df_profile['Accident_Severity'].replace({'Fatal': 'Fatal'})
df_profile['date_parsed'] = pd.to_datetime(df_profile['Accident Date'], format='%d-%m-%Y', errors='coerce')
df_profile['hour'] = pd.to_datetime(df_profile['Time'], format='%H:%M', errors='coerce').dt.hour

profile = {
    'original_rows': int(original_rows),
    'original_columns': int(original_columns),
    'original_column_names': original_column_names,
    'columns_after_profiling': int(df_profile.shape[1]),
    'profiling_added_columns': [
        'Accident_Severity_Clean',
        'date_parsed',
        'hour'
    ]
}
```

```

... {'original_rows': 307973,
      'original_columns': 21,
      'original_column_names': ['Accident_Index',
                                'Accident Date',
                                'Day_of_Week',
                                'Junction_Control',
                                'Junction_Detail',
                                'Accident_Severity',
                                'Latitude',
                                'Light_Conditions',
                                'Local_Authority_(District)',
                                'Carriageway_Hazards',
                                'Longitude',
                                'Number_of_Casualties',
                                'Number_of_Vehicles',
                                'Police_Force',
                                'Road_Surface_Conditions',
                                'Road_Type',
                                'Speed_limit',
                                'Time',
                                'Urban_or_Rural_Area',
                                'Weather_Conditions',
                                'Vehicle_Type'],
      'columns_after_profiling': 24,
      'profiling_added_columns': ['Accident_Severity_Clean', 'date_parsed', 'hour'],
      'duplicate_rows': 0,
      'missing_values_original': {'Carriageway_Hazards': 302549,
                                   'Time': 17,
                                   'Day_of_Week': 0,
                                   'Accident Date': 0,

```

Gambar 5. Hasil Dataset Profilling

4.3 Apache Spark

Pada tahap ini, Apache Spark dijalankan di Google Colab dengan membuat Spark Session. Spark Session digunakan sebagai penghubung utama agar sistem dapat membaca dan memproses dataset menggunakan Spark. Spark Session dibuat dengan nama RoadAccidentUK_BigData_Colab. Setelah Spark berhasil aktif, data kecelakaan dapat diproses untuk tahap berikutnya, seperti cleaning, preprocessing, analisis, dan clustering K-Means.

```

▶ # FASE 4 - Spark Session
# 1. Mengaktifkan Apache Spark di Google Colab
# 2. Membuat objek spark agar bisa membaca dan memproses dataset besar

from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
        .appName('RoadAccidentUK_BigData_Colab')
        .config('spark.sql.shuffle.partitions', '8')
        .getOrCreate()
)

spark

... SparkSession - in-memory
SparkContext
Spark UI
Version
v4.0.2
Master
local[*]
AppName
RoadAccidentUK_BigData_Colab

```

Gambar 6. Implementasi Spark Session pada Google Colab

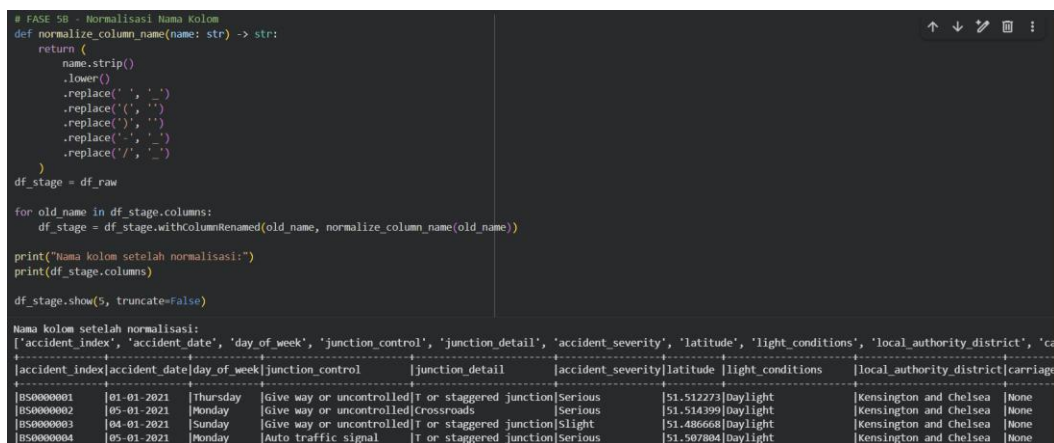
4.4 Cleaning dan Preprocessing Data

Pada tahap ini dilakukan untuk memperbaiki kondisi data sebelum digunakan pada tahap analisis. Proses ini mencakup normalisasi nama kolom, pemeriksaan masalah data, pembersihan data kategorikal, penyesuaian tanggal dan waktu, pembuatan fitur baru, validasi data, serta finalisasi data bersih.

4.4.1 Normalisasi Nama Kolom

Pada tahap ini, nama kolom pada dataset dinormalisasi agar lebih mudah digunakan dalam proses pengolahan data. Nama kolom yang awalnya memiliki spasi, tanda kurung, tanda hubung, garis miring, atau huruf besar diubah menjadi format yang lebih sederhana dan konsisten. Contohnya, kolom Accident Date diubah menjadi accident_date, sedangkan Local_Authority_(District) diubah menjadi local_authority_district.

Hasil dari proses ini adalah seluruh nama kolom memiliki format yang lebih rapi, yaitu menggunakan huruf kecil dan tanda underscore. Dengan format tersebut, pemanggilan kolom pada tahap cleaning, preprocessing, analisis, dan penyimpanan data menjadi lebih mudah dilakukan.



```
# FASE 5B - Normalisasi Nama Kolom
def normalize_column_name(name: str) -> str:
    return (
        name.strip()
        .lower()
        .replace(' ', '_')
        .replace('(', '_')
        .replace(')', '_')
        .replace('-', '_')
        .replace('/', '_')
    )

df_stage = df_raw

for old_name in df_stage.columns:
    df_stage = df_stage.withColumnRenamed(old_name, normalize_column_name(old_name))

print("Nama kolom setelah normalisasi:")
print(df_stage.columns)

df_stage.show(5, truncate=False)

Nama kolom setelah normalisasi:
['accident_index', 'accident_date', 'day_of_week', 'junction_control', 'junction_detail', 'accident_severity', 'latitude', 'light_conditions', 'local_authority_district', 'carriage']
```

accident_index	accident_date	day_of_week	junction_control	junction_detail	accident_severity	latitude	light_conditions	local_authority_district	carriage
[850000001]	[01-01-2021]	[Thursday]	[Give way or uncontrolled]	[T or staggered junction]	[Serious]	[51.512273]	[Daylight]	[Kensington and Chelsea]	[None]
[850000002]	[05-01-2021]	[Monday]	[Give way or uncontrolled]	[Crossroads]	[Serious]	[51.514399]	[Daylight]	[Kensington and Chelsea]	[None]
[850000003]	[04-01-2021]	[Sunday]	[Give way or uncontrolled]	[T or staggered junction]	[Slight]	[51.486668]	[Daylight]	[Kensington and Chelsea]	[None]
[850000004]	[05-01-2021]	[Monday]	[Auto traffic signal]	[T or staggered junction]	[Serious]	[51.507804]	[Daylight]	[Kensington and Chelsea]	[None]

Gambar 7. Normalisasi Nama Kolom Dataset

4.4.2 Pemeriksaan Masalah Data

Pada tahap ini, sistem melakukan pemeriksaan awal untuk mengetahui masalah yang terdapat pada dataset. Pemeriksaan dilakukan terhadap missing value, nilai placeholder seperti None atau Unknown, data duplikat, kesalahan penulisan, format tanggal dan waktu, koordinat, nilai numerik, serta spasi berlebih pada data teks.

Hasil pemeriksaan menunjukkan bahwa dataset tidak memiliki duplikasi pada seluruh baris data. Namun, terdapat beberapa masalah yang perlu ditangani, seperti missing value pada kolom tertentu, typo pada kategori accident_severity, serta beberapa data waktu yang kosong atau tidak valid. Selain itu, nilai koordinat dan numerik juga diperiksa untuk memastikan data masih masuk akal sebelum digunakan pada analisis dan clustering.

Hasil dari tahap ini menjadi dasar untuk menentukan proses cleaning yang dilakukan pada tahap berikutnya. Dengan pemeriksaan ini, data yang bermasalah dapat diketahui lebih awal sehingga proses pengolahan data menjadi lebih terarah.

```
# FASE 5C.2 - Deteksi Duplikasi Data

duplicate_all_rows = total_rows - df_stage.dropDuplicates().count()

duplicate_accident_index_df = (
    df_stage
    .groupBy("accident_index")
    .count()
    .filter(col("count") > 1)
)

duplicate_accident_index_count = duplicate_accident_index_df.count()

print("Jumlah duplikasi seluruh baris:", duplicate_all_rows)
print("Jumlah accident_index yang duplikat:", duplicate_accident_index_count)

print("Contoh accident_index duplikat jika ada:")
duplicate_accident_index_df.show(10, truncate=False)

noise_report["duplicate_all_rows"] = duplicate_all_rows
noise_report["duplicate_accident_index_count"] = duplicate_accident_index_count

Jumlah duplikasi seluruh baris: 0
Jumlah accident_index yang duplikat: 0
Contoh accident_index duplikat jika ada:
+-----+-----+
|accident_index|count|
+-----+-----+
+-----+-----+
```

Gambar 8. Hasil Pemeriksaan Data Duplikat

Kolom dengan missing value tertinggi:

	missing_count	missing_percentage
time	17	0.01
carriageway_hazards	3	0.00
day_of_week	0	0.00
accident_date	0	0.00
accident_index	0	0.00
junction_detail	0	0.00
junction_control	0	0.00
light_conditions	0	0.00
accident_severity	0	0.00
local_authority_district	0	0.00

Kesimpulan awal:

1. Kolom time memiliki sejumlah data kosong atau tidak valid.
2. Kolom carriageway_hazards memiliki sedikit missing value aktual, sedangkan nilai None dapat dimaknai sebagai tidak ada hazard.
3. Jika terdapat typo Fatal pada accident_severity, maka akan diperbaiki menjadi Fatal.
4. Kolom teks perlu dirapikan menggunakan trim.
5. Koordinat dan nilai numerik perlu diberi flag validitas untuk kebutuhan analisis spasial dan clustering.

Gambar 9. Ringkasan Masalah Data

4.4.3 Cleaning Data Kategorikal dan Waktu

Pada tahap ini, sistem membersihkan data kategorikal dan data waktu agar lebih rapi dan konsisten. Data kategorikal dibersihkan dengan menghapus spasi berlebih, memperbaiki penulisan kategori, serta mengisi nilai kosong pada beberapa kolom. Salah satu perbaikan yang dilakukan adalah mengubah typo Fetal pada kolom accident_severity menjadi Fatal.

Selain itu, data waktu juga disesuaikan agar dapat digunakan dalam proses analisis. Kolom time dibersihkan dan disesuaikan ke format jam, sedangkan kolom accident_date diubah menjadi format tanggal. Dari proses ini, sistem juga membuat beberapa atribut waktu baru, seperti accident_year, accident_month, accident_day, dan hour.

Hasil dari tahap ini membuat data kategorikal menjadi lebih seragam dan data waktu dapat digunakan untuk analisis berdasarkan tahun, bulan, hari, dan jam kejadian. Dengan begitu, data sudah lebih siap digunakan pada tahap analisis tren kecelakaan dan visualisasi dashboard.

```
Tipe data kolom time sebelum cleaning: timestamp
Nilai time yang digunakan untuk imputasi: 17:00
Cek hasil fitur waktu setelah cleaning dan imputasi:
```

accident_date	accident_date_parsed	accident_year	accident_month	accident_day	time	hour
01-01-2021	2021-01-01	2021	1	1	15:11	15
05-01-2021	2021-01-05	2021	1	5	10:59	10
04-01-2021	2021-01-04	2021	1	4	14:19	14
05-01-2021	2021-01-05	2021	1	5	08:10	8
06-01-2021	2021-01-06	2021	1	6	17:25	17
01-01-2021	2021-01-01	2021	1	1	11:48	11
08-01-2021	2021-01-08	2021	1	8	13:58	13
02-01-2021	2021-01-02	2021	1	2	13:18	13
07-01-2021	2021-01-07	2021	1	7	12:15	12
10-01-2021	2021-01-10	2021	1	10	09:52	9

```
only showing top 10 rows
Jumlah time yang masih kosong:
0
Jumlah hour yang masih kosong:
0
```

Gambar 10. Hasil Cleaning Data Waktu

4.4.4 Feature Engineering

Feature engineering dilakukan untuk membuat fitur baru yang dapat membantu proses analisis data. Pada tahap ini, sistem membuat beberapa kolom tambahan, yaitu `is_weekend`, `severity_rank`, dan `high_severity_flag`. Kolom `is_weekend` digunakan untuk membedakan kecelakaan yang terjadi pada hari kerja dan akhir pekan. Kolom `severity_rank` digunakan untuk mengubah tingkat keparahan kecelakaan menjadi nilai numerik, yaitu Slight, Serious, dan Fatal. Sementara itu, `high_severity_flag` digunakan untuk menandai kecelakaan dengan tingkat keparahan tinggi, yaitu kategori Serious dan Fatal.

Hasil dari tahap ini membuat dataset lebih siap digunakan untuk analisis lanjutan. Fitur baru tersebut membantu sistem dalam melihat pola kecelakaan berdasarkan akhir pekan, tingkat keparahan, dan kecelakaan dengan risiko tinggi.

```
Cek fitur severity dan weekend:
```

day_of_week	is_weekend	accident_severity	severity_rank	high_severity_flag
Thursday	0	Serious	2	1
Monday	0	Serious	2	1
Sunday	1	Slight	1	0
Monday	0	Serious	2	1
Tuesday	0	Serious	2	1
Thursday	0	Slight	1	0
Thursday	0	Serious	2	1
Friday	0	Slight	1	0
Wednesday	0	Slight	1	0
Saturday	1	Slight	1	0
Wednesday	0	Slight	1	0
Friday	0	Slight	1	0
Monday	0	Slight	1	0
Friday	0	Slight	1	0
Saturday	1	Slight	1	0
Sunday	1	Serious	2	1
Monday	0	Slight	1	0
Monday	0	Slight	1	0
Monday	0	Slight	1	0
Tuesday	0	Slight	1	0

```
only showing top 20 rows
Distribusi weekend vs weekday:
```

is_weekend	count
1	75110
0	232863

```
Distribusi high severity:
```

high_severity_flag	count
1	44691
0	263280

Gambar 11. Hasil Feature Engineering

4.4.5 Validasi dan Finalisasi Data Bersih

Pada tahap ini, sistem melakukan validasi terhadap data yang sudah dibersihkan. Validasi dilakukan pada koordinat lokasi dan nilai numerik. Koordinat dicek agar tetap berada dalam rentang wilayah Inggris, sedangkan nilai numerik seperti jumlah korban, jumlah kendaraan, dan batas kecepatan diperiksa agar tetap logis.

Setelah proses validasi selesai, sistem memilih kolom-kolom yang relevan dan menghapus duplikasi berdasarkan `accident_index`. Hasil akhir dari tahap ini adalah dataframe bersih bernama `df_clean`. Data ini sudah siap digunakan untuk analisis, clustering K-Means, penyimpanan ke database, dan visualisasi dashboard.

```
Distribusi severity setelah preprocessing:
+-----+
|accident_severity| count|
+-----+
| Slight|263288|
| Serious| 48748|
| Fatal| 3953|
+-----+

Jumlah typo Fatal setelah preprocessing:
0
Jumlah time kosong setelah preprocessing:
0
Jumlah hour kosong setelah preprocessing:
0
Distribusi coordinate_valid:
+-----+
|coordinate_valid| count|
+-----+
| 1|387973|
+-----+

Distribusi numeric_valid:
+-----+
|numeric_valid| count|
+-----+
| 1|387973|
+-----+

Cek contoh data final:
+-----+-----+-----+-----+-----+-----+-----+-----+
|accident_index|accident_date|hour|is_weekend|accident_severity|severity_rank|high_severity_flag|coordinate_valid|numeric_valid|
+-----+-----+-----+-----+-----+-----+-----+-----+
|850000002|05-01-2021|10|0|Serious|2|1|1|1|
|850000003|04-01-2021|14|1|Slight|1|0|1|1|
|850000006|01-01-2021|11|0|Slight|1|0|1|1|
|850000009|07-01-2021|12|0|Slight|1|0|1|1|
|850000011|07-01-2021|0|0|Slight|1|0|1|1|
|850000013|12-01-2021|14|0|Slight|1|0|1|1|
|850000015|17-01-2021|12|1|Slight|1|0|1|1|
|850000016|25-01-2021|22|1|Serious|2|1|1|1|
|850000018|26-01-2021|17|0|Slight|1|0|1|1|
|850000020|27-01-2021|0|0|Slight|1|0|1|1|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows
```

Gambar 12. Cek Kualitas Data Setelah Preprocessing

4.6 Analisis Data

Pada tahap ini, data bersih yang sudah melalui proses preprocessing dimuat kembali untuk dianalisis menggunakan Apache Spark. Data tersebut dibaca dari file `accident_clean.parquet` agar proses analisis dapat dilakukan dengan lebih cepat dan terstruktur.

Analisis dilakukan untuk menghasilkan beberapa ringkasan data, seperti jumlah kecelakaan berdasarkan tingkat keparahan, tren kecelakaan per jam, tren kecelakaan per bulan, perbandingan weekday dan weekend, district dengan jumlah kecelakaan tertinggi, jenis kendaraan, kondisi cuaca, kondisi jalan, batas kecepatan, dan ringkasan lokasi awal untuk hotspot kecelakaan.

Hasil dari tahap ini berupa beberapa output analisis, seperti `severity_summary`, `hourly_trend`, `monthly_trend`, `weekend_vs_weekday`, `district_summary`, `vehicle_summary`, `weather_by_severity`, `road_surface_by_severity`, `speed_limit_summary`, dan

spatial_grid_summary. Output tersebut kemudian digunakan pada tahap penyimpanan database dan visualisasi dashboard.

```
# FASE 7C - Cek Output Analisis Spark

from pathlib import Path

output_folders = [
    "severity_summary",
    "hourly_trend",
    "monthly_trend",
    "weekend_vs_weekday",
    "district_summary",
    "vehicle_summary",
    "weather_by_severity",
    "road_surface_by_severity",
    "speed_limit_summary",
    "spatial_grid_summary"
]

print("Cek folder output analisis Spark:")

for folder in output_folders:
    path = OUTPUT_DIR / folder
    print(folder, "->", path.exists())
```

```
Cek folder output analisis Spark:
severity_summary -> True
hourly_trend -> True
monthly_trend -> True
weekend_vs_weekday -> True
district_summary -> True
vehicle_summary -> True
weather_by_severity -> True
road_surface_by_severity -> True
speed_limit_summary -> True
spatial_grid_summary -> True
```

Gambar 13. Verifikasi Output Analisis Spark

4.7 Clustering Hotspot dengan K-Means

Pada tahap ini, sistem melakukan clustering hotspot kecelakaan menggunakan algoritma K-Means dari Spark MLlib. Data yang digunakan adalah data yang sudah melalui proses preprocessing dan memiliki koordinat serta nilai numerik yang valid.

Atribut yang digunakan dalam proses clustering meliputi latitude, longitude, speed_limit, number_of_casualties, dan number_of_vehicles. Atribut tersebut digabungkan menjadi satu fitur vektor, kemudian distandardisasi agar setiap atribut memiliki skala yang seimbang sebelum diproses oleh K-Means.

Pada proyek ini, jumlah cluster ditentukan sebanyak 10 cluster. Hasil clustering menghasilkan cluster_id untuk setiap data kecelakaan. Selain itu, sistem juga membuat ringkasan setiap cluster, seperti jumlah kecelakaan, titik pusat latitude dan longitude, serta nilai high_severity_rate. Hasil ini kemudian disimpan sebagai hotspot_predictions dan hotspot_cluster_summary untuk digunakan pada tahap analisis, database, dan visualisasi dashboard.

```

# FASE 8 - Hotspot Clustering dengan Spark MLlib

from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.clustering import KMeans
from pyspark.sql.functions import col, count, avg, round as spark_round

# Mengambil data yang koordinat dan numeriknya valid
cluster_df = (
    df_analysis
    .filter(col("coordinate_valid") == 1)
    .filter(col("numeric_valid") == 1)
    .filter(col("latitude").isNotNull() & col("longitude").isNotNull())
    .select(
        "accident_index",
        "latitude",
        "longitude",
        "speed_limit",
        "number_of_casualties",
        "number_of_vehicles",
        "high_severity_flag",
        "accident_severity",
        "local_authority_district"
    )
)

print("Jumlah data untuk clustering:", cluster_df.count())

# Menggabungkan fitur numerik ke dalam satu vector
assembler = VectorAssembler(
    inputCols=[
        "latitude",
        "longitude",
        "speed_limit",
        "number_of_casualties",
        "number_of_vehicles"
    ],
    outputCol="features_raw",
    handleInvalid="skip"
)

```

Gambar 14. Clustering Hotspot dengan K-Means

```

Jumlah data untuk clustering: 307973
Training cost: 570481.315148754
Ringkasan cluster hotspot:
+-----+-----+-----+-----+-----+
|cluster_id|total_accidents|center_latitude|center_longitude|high_severity_rate|
+-----+-----+-----+-----+-----+
|9         |83434         |53.414929     |-1.881707      |0.1305            |
|4         |79617         |51.539697     |-0.14381       |0.1366            |
|0         |27680         |51.879887     |-0.149534      |0.1818            |
|3         |25993         |52.483767     |-2.404222      |0.1734            |
|1         |24108         |51.206091     |-3.207963      |0.1087            |
|7         |21762         |51.769006     |-0.387042      |0.1102            |
|2         |15219         |52.845486     |-1.760697      |0.2227            |
|8         |12522         |52.421119     |-1.487115      |0.145             |
|5         |9662          |55.472488     |-2.682806      |0.2059            |
|6         |7976          |55.987432     |-3.686042      |0.1477            |
+-----+-----+-----+-----+-----+

```

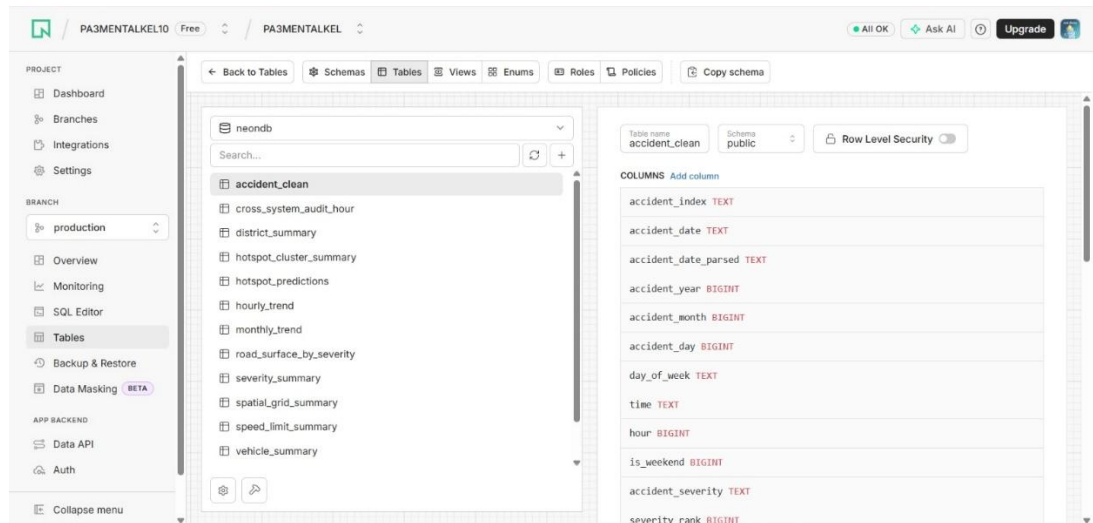
Gambar 15. Hasil Clustering Hotspot dengan K-Means

4.8 Penyimpanan Data ke PostgreSQL

Pada tahap ini, data yang sudah selesai diproses disimpan ke PostgreSQL. Database PostgreSQL yang digunakan berjalan melalui layanan Neon, sedangkan koneksi dari Google Colab ke PostgreSQL dilakukan menggunakan SQLAlchemy dan psycopg2. Koneksi dilakukan dengan memasukkan connection string dari Neon ke dalam Google Colab. Setelah itu, sistem menjalankan query `SELECT version()` untuk memastikan bahwa koneksi ke database berhasil.

Setelah koneksi berhasil, file CSV hasil preprocessing dan hasil analisis diunggah ke PostgreSQL dalam bentuk tabel. Data utama disimpan pada tabel `accident_clean`, sedangkan hasil analisis disimpan dalam tabel ringkasan seperti `severity_summary`, `hourly_trend`, `monthly_trend`, `district_summary`, `vehicle_summary`, `speed_limit_summary`, dan `hotspot_cluster_summary`.

Setelah proses upload selesai, sistem melakukan pengecekan jumlah baris pada setiap tabel. Pengecekan ini dilakukan untuk memastikan bahwa data sudah berhasil masuk ke PostgreSQL dan dapat digunakan kembali untuk query analisis maupun proses integrasi ke MongoDB.



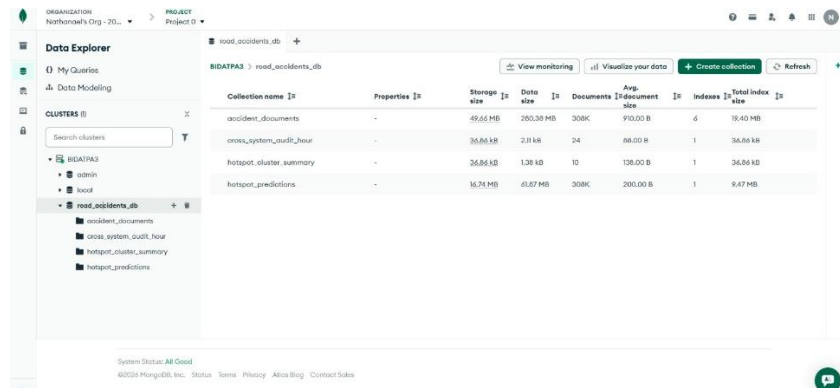
Gambar 16. Tampilan Tabel pada Neon PostgreSQL

4.9 Penyimpanan Data ke MongoDB

Pada tahap ini, data hasil preprocessing dan hasil clustering disimpan ke MongoDB Atlas sebagai database NoSQL. Koneksi dari Google Colab ke MongoDB dilakukan menggunakan library `pymongo`. Setelah koneksi berhasil, sistem menggunakan database `road_accidents_db` sebagai tempat penyimpanan data.

Data utama dari file `accident_clean.csv` dimasukkan ke collection `accident_documents`. Data tersebut disimpan dalam bentuk dokumen bertingkat atau nested document, sehingga informasi seperti severity, lokasi, waktu, kondisi jalan, kendaraan, dan jumlah korban dapat dikelompokkan dengan lebih rapi. Selain itu, hasil clustering juga disimpan ke collection `hotspot_cluster_summary` dan `hotspot_predictions`.

Setelah proses upload selesai, sistem melakukan verifikasi dengan menghitung jumlah dokumen pada setiap collection dan menampilkan contoh dokumen. Verifikasi ini dilakukan untuk memastikan bahwa data berhasil tersimpan di MongoDB dan siap digunakan pada tahap integrasi maupun visualisasi dashboard.



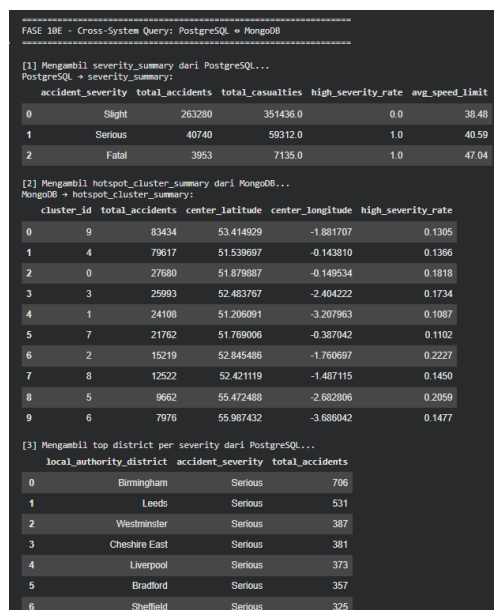
Gambar 17. Tampilan Collection pada MongoDB Atlas

4.10 Integrasi PostgreSQL ke MongoDB

Pada tahap ini, sistem melakukan integrasi antara PostgreSQL dan MongoDB. Integrasi dilakukan dengan mengambil hasil query dari PostgreSQL, kemudian memindahkannya ke MongoDB agar data dapat digunakan kembali dalam bentuk dokumen.

Data yang diambil adalah ringkasan kecelakaan berdasarkan local_authority_district. Query tersebut menghasilkan informasi seperti nama district, total kecelakaan, total korban, dan tingkat kecelakaan berat. Hasil query kemudian dibaca menggunakan pandas, diubah ke format dictionary, lalu dimasukkan ke collection MongoDB bernama district_summary_from_postgresql.

Hasil dari tahap ini menunjukkan bahwa data dari PostgreSQL berhasil dipindahkan ke MongoDB. Dengan demikian, sistem sudah memenuhi proses integrasi SQL dan NoSQL, karena data dapat berpindah dari database relasional ke database NoSQL.



Gambar 18. Hasil Integrasi PostgreSQL dan MongoDB

4.11 Dashboard Visualisasi

Pada tahap ini, dashboard visualisasi dibuat menggunakan Streamlit dan Plotly. Dashboard digunakan untuk menampilkan hasil analisis data kecelakaan dalam bentuk grafik, tabel, dan insight singkat agar lebih mudah dipahami oleh pengguna.

Data yang ditampilkan pada dashboard berasal dari hasil pengolahan dan penyimpanan database. Dashboard menampilkan beberapa informasi utama, seperti total kecelakaan, total korban, tingkat keparahan kecelakaan, district dengan jumlah kecelakaan tertinggi, serta ringkasan data lainnya.

Dashboard dijalankan melalui Google Colab dengan bantuan ngrok agar dapat diakses melalui browser. Setelah dashboard berhasil dijalankan, sistem menampilkan link publik yang dapat digunakan untuk membuka tampilan dashboard.

```
✓ Dashboard disalin ke /content/dashboard_streamlit.py
✓ Streamlit berjalan di port 8501 (PID: 81672)
WARNING:pyngrok.process.ngrok:t=2026-05-28T06:26:39+0000 lvl=warn msg="can't bind default web address, trying alternatives" obj=web addr=127.0.0.1:4040

=====
! DASHBOARD SIAP DIAKSES!
=====
🔗 Link: https://freeload-blunt-bobcat.ngrok-free.dev
```

Gambar 19. Proses Menjalankan Dashboard Streamlit melalui Ngrok

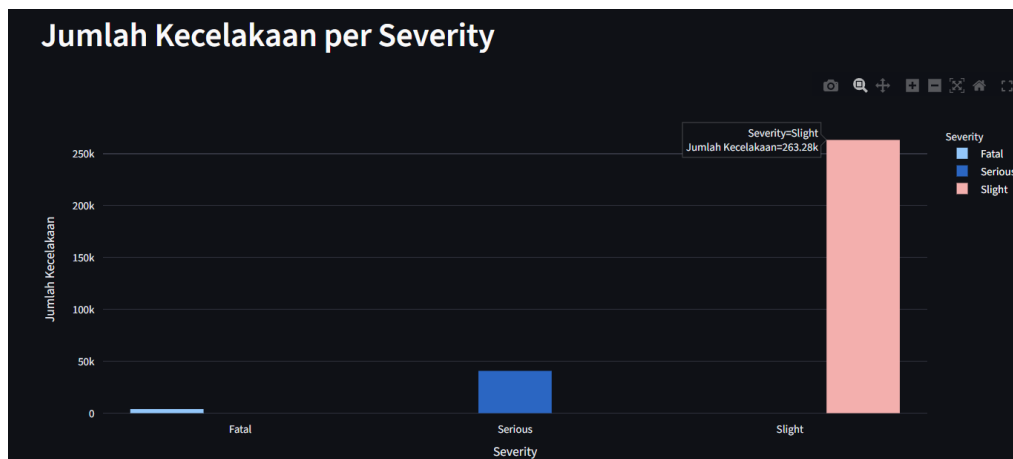
BAB V

HASIL DAN ANALISIS

5.1 Analisis Tingkat Keparahan Kecelakaan (*Severity Analysis*)

Berdasarkan hasil analisis, tingkat keparahan kecelakaan dibagi menjadi tiga kategori, yaitu Slight, Serious, dan Fatal. Hasil analisis menunjukkan bahwa kategori Slight memiliki jumlah kecelakaan paling tinggi, yaitu 263.280 kejadian. Kategori Serious berada di urutan kedua dengan 40.740 kejadian, sedangkan kategori Fatal memiliki jumlah paling rendah, yaitu 3.953 kejadian.

Hasil ini menunjukkan bahwa sebagian besar kecelakaan yang terjadi termasuk dalam kategori ringan. Meskipun begitu, kategori Serious dan Fatal tetap perlu diperhatikan karena berhubungan langsung dengan kecelakaan yang memiliki dampak lebih besar terhadap korban.

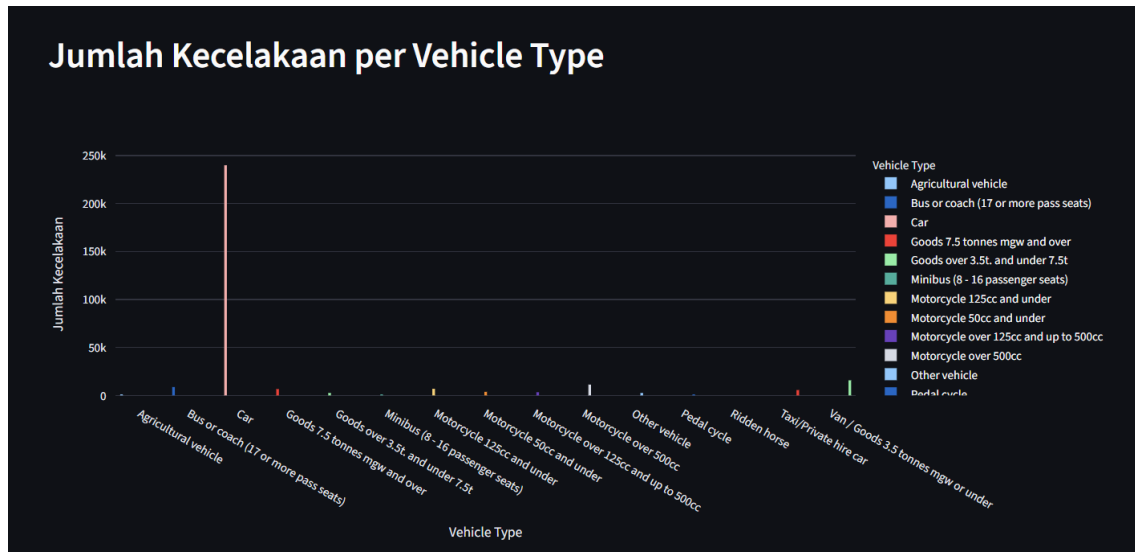


Gambar 20. Jumlah Kecelakaan Berdasarkan Tingkat Keparahan

5.2 Analisis Jenis Kendaraan (*Vehicle Type Analysis*)

Visualisasi ini menampilkan jumlah kecelakaan berdasarkan jenis kendaraan. Dari grafik yang ditampilkan, Car menjadi jenis kendaraan dengan jumlah kecelakaan paling tinggi. Jenis kendaraan lain seperti van, motorcycle, bus, goods vehicle, dan pedal cycle memiliki jumlah yang lebih kecil dibandingkan mobil.

Hasil ini menunjukkan bahwa mobil menjadi kendaraan yang paling sering muncul dalam data kecelakaan. Hal ini dapat terjadi karena mobil merupakan kendaraan yang banyak digunakan dalam aktivitas harian. Meskipun demikian, jenis kendaraan lain tetap perlu diperhatikan, terutama motorcycle dan goods vehicle, karena karakteristik risikonya berbeda. Motorcycle cenderung lebih rentan terhadap korban serius, sedangkan goods vehicle berkaitan dengan ukuran kendaraan dan potensi dampak kecelakaan yang lebih besar.

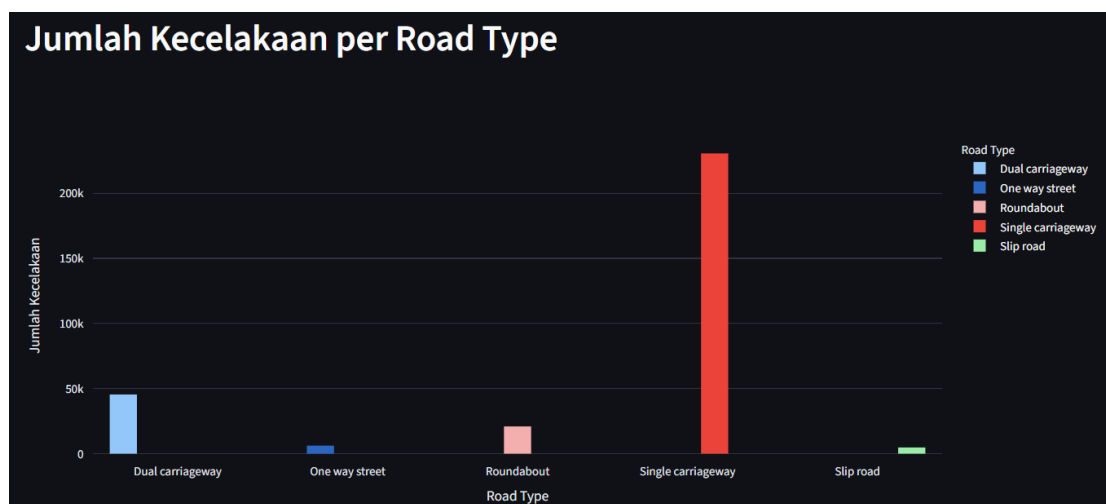


Gambar 21. Jumlah Kecelakaan Berdasarkan Vehicle Type

5.3 Analisis Jumlah Kecelakaan Berdasarkan Road Type

Visualisasi road type menunjukkan bahwa Single carriageway memiliki jumlah kecelakaan paling tinggi dibandingkan jenis jalan lainnya. Setelah itu terdapat dual carriageway, roundabout, one way street, dan slip road dengan jumlah yang lebih rendah.

Hasil ini menunjukkan bahwa single carriageway menjadi jenis jalan yang paling dominan dalam data kecelakaan. Jalan jenis ini biasanya memiliki arus lalu lintas dua arah tanpa pemisah fisik yang kuat seperti pada dual carriageway. Kondisi tersebut dapat meningkatkan potensi konflik kendaraan, terutama ketika volume lalu lintas padat, kendaraan berbelok, atau pengemudi menyalip. Namun, hasil ini tidak langsung berarti single carriageway paling berbahaya. Jumlah kecelakaan juga dapat dipengaruhi oleh seberapa banyak jalan jenis tersebut digunakan.

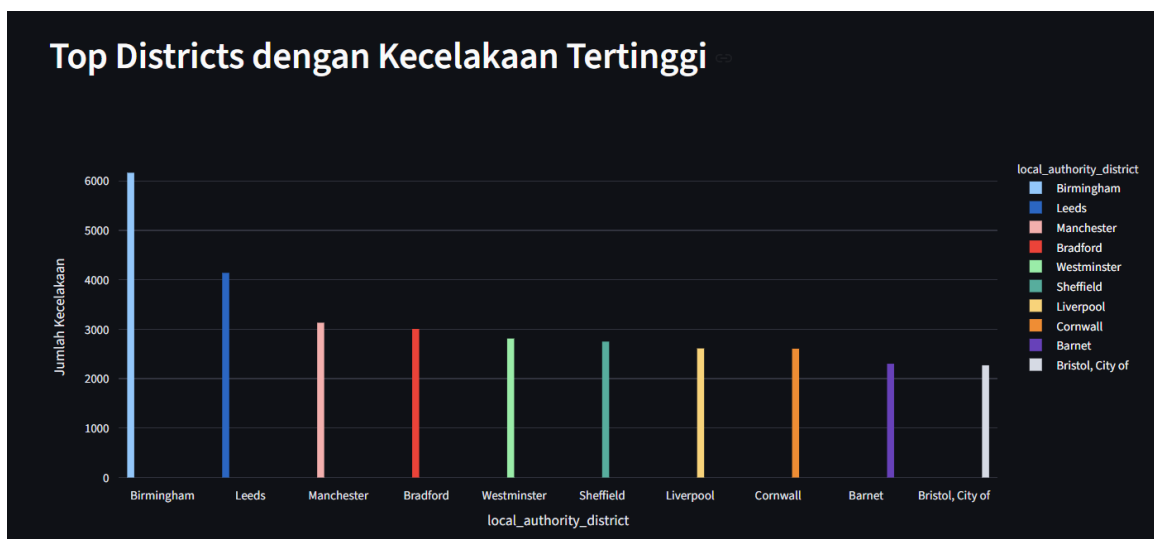


Gambar 22. Jumlah Kecelakaan Berdasarkan Road Type

5.4 Analisis Top District dengan Kecelakaan Tertinggi

Visualisasi top district menampilkan 10 district dengan jumlah kecelakaan tertinggi. Berdasarkan grafik, Birmingham menjadi district dengan jumlah kecelakaan paling tinggi. Setelah itu terdapat Leeds, Manchester, Bradford, Westminster, Sheffield, Liverpool, Cornwall, Barnet, dan Bristol City.

Hasil ini menunjukkan bahwa kecelakaan lebih banyak terjadi pada district besar atau wilayah dengan aktivitas transportasi yang tinggi. District seperti Birmingham, Leeds, dan Manchester merupakan wilayah perkotaan yang memiliki mobilitas tinggi, sehingga peluang terjadinya kecelakaan juga lebih besar. Visualisasi ini penting karena dapat digunakan untuk menentukan wilayah prioritas dalam pengawasan keselamatan jalan.

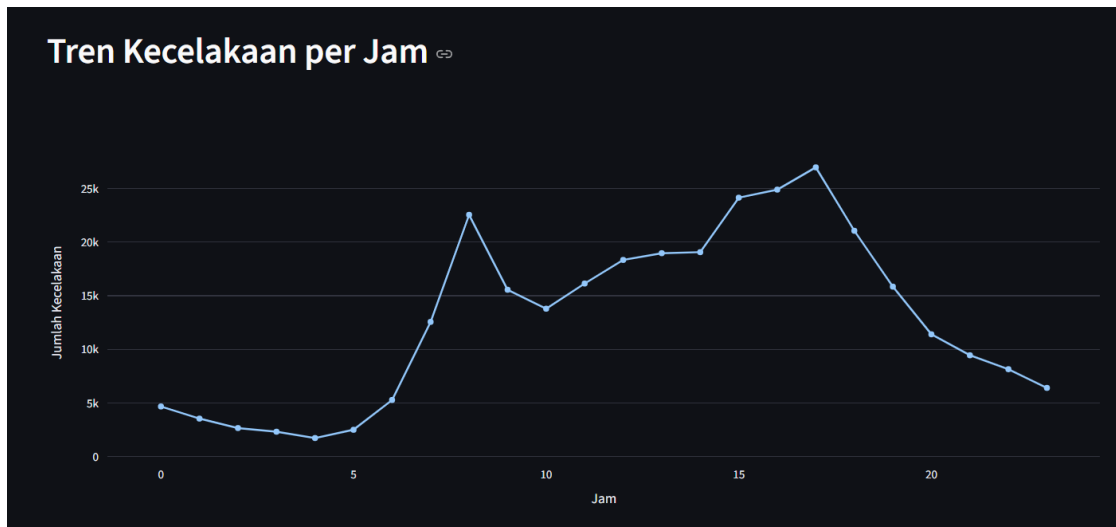


Gambar 23. Top District dengan Kecelakaan Tertinggi

5.5 Analisis Tren Kecelakaan per Jam

Grafik tren kecelakaan per jam menunjukkan bahwa jumlah kecelakaan cenderung rendah pada dini hari. Jumlah kecelakaan mulai meningkat pada pagi hari, terutama sekitar jam berangkat kerja atau sekolah. Setelah itu, jumlah kecelakaan kembali meningkat dan mencapai puncaknya pada sore hari, terutama sekitar pukul 16.00 sampai 17.00.

Pola ini menunjukkan bahwa waktu dengan aktivitas lalu lintas tinggi cenderung memiliki jumlah kecelakaan lebih besar. Pagi hari berkaitan dengan mobilitas awal hari, sedangkan sore hari berkaitan dengan jam pulang kerja, pulang sekolah, dan peningkatan kepadatan jalan. Dengan demikian, waktu sore dapat menjadi periode yang perlu mendapat perhatian lebih dalam pengelolaan keselamatan lalu lintas.

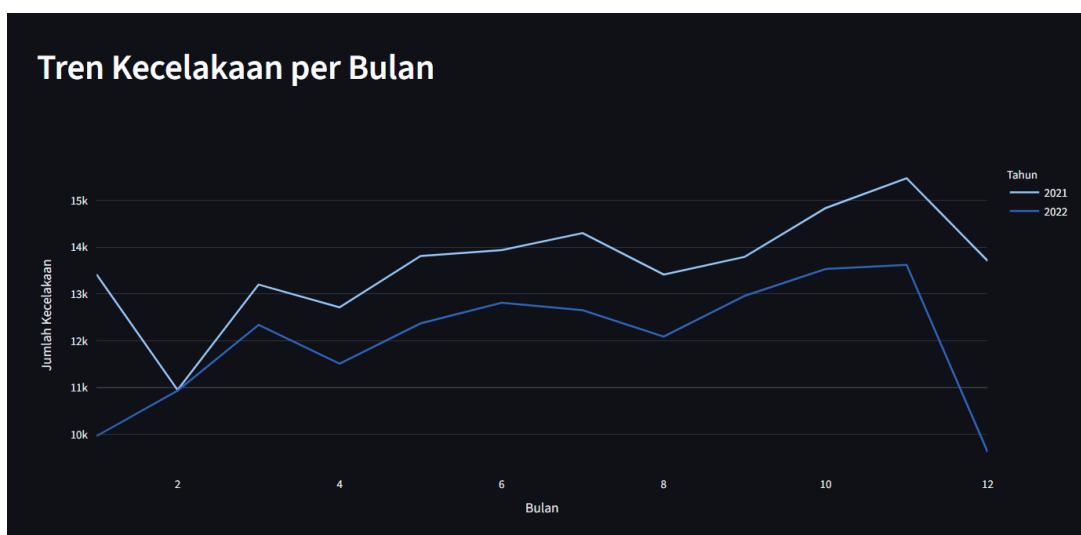


Gambar 24. Tren Kecelakaan per Jam

5.6 Analisis Tren Kecelakaan per Bulan

Visualisasi tren kecelakaan per bulan membandingkan jumlah kecelakaan pada tahun 2021 dan 2022. Secara umum, data tahun 2021 terlihat lebih tinggi dibandingkan tahun 2022 pada beberapa bulan. Jumlah kecelakaan juga cenderung meningkat pada pertengahan sampai akhir tahun, terutama mendekati bulan Oktober dan November.

Pada tahun 2022, grafik menunjukkan penurunan tajam pada bulan Desember. Penurunan ini perlu dibaca dengan hati-hati karena dapat disebabkan oleh cakupan data yang belum penuh, perbedaan jumlah record, atau kondisi tertentu pada periode tersebut. Oleh karena itu, tren bulanan tidak hanya dibaca sebagai pola kecelakaan, tetapi juga perlu dikaitkan dengan kelengkapan data per bulan.

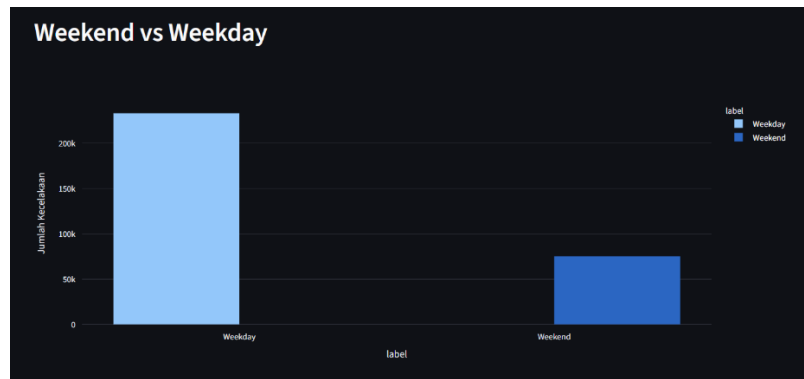


Gambar 25. Tren Kecelakaan per Bulan

5.7 Analisis Perbandingan Weekend dan Weekday

Visualisasi weekend vs weekday menunjukkan bahwa jumlah kecelakaan pada weekday jauh lebih tinggi dibandingkan weekend. Hal ini dapat terjadi karena weekday terdiri dari lima hari kerja, sedangkan weekend hanya dua hari. Selain itu, aktivitas harian seperti bekerja, sekolah, distribusi barang, dan perjalanan rutin lebih banyak terjadi pada weekday.

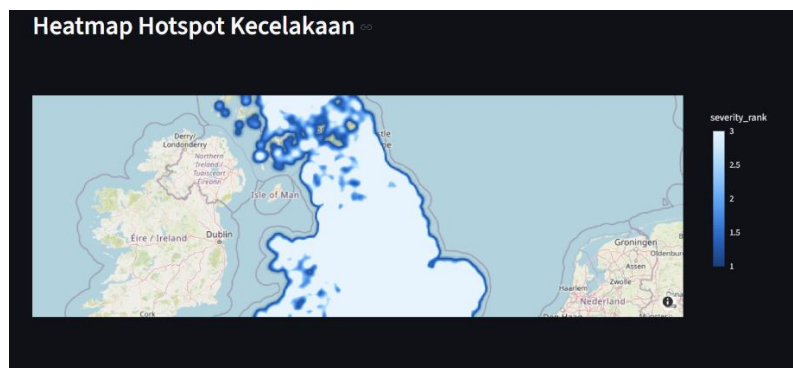
Namun, hasil ini belum tentu menunjukkan bahwa weekday lebih berisiko secara mutlak. Agar lebih adil, analisis lanjutan dapat menghitung rata-rata kecelakaan per hari. Dengan begitu, perbandingan weekday dan weekend tidak hanya dilihat dari total jumlah kecelakaan, tetapi juga dari tingkat kejadian rata-rata.



Gambar 26. Perbandingan Weekend dan Weekday

5.8 Analisis Heatmap Hotspot Kecelakaan

Heatmap hotspot kecelakaan digunakan untuk melihat sebaran kecelakaan berdasarkan latitude dan longitude. Pada visualisasi ini, kepadatan kecelakaan ditampilkan menggunakan peta, sedangkan nilai severity_rank digunakan sebagai bobot tingkat keparahan. Dengan cara ini, area yang memiliki kepadatan kecelakaan tinggi dan tingkat severity lebih besar dapat terlihat lebih jelas. Visualisasi heatmap membantu pengguna memahami lokasi yang berpotensi menjadi titik rawan kecelakaan. Titik dengan warna yang lebih kuat menunjukkan konsentrasi kecelakaan yang lebih tinggi.

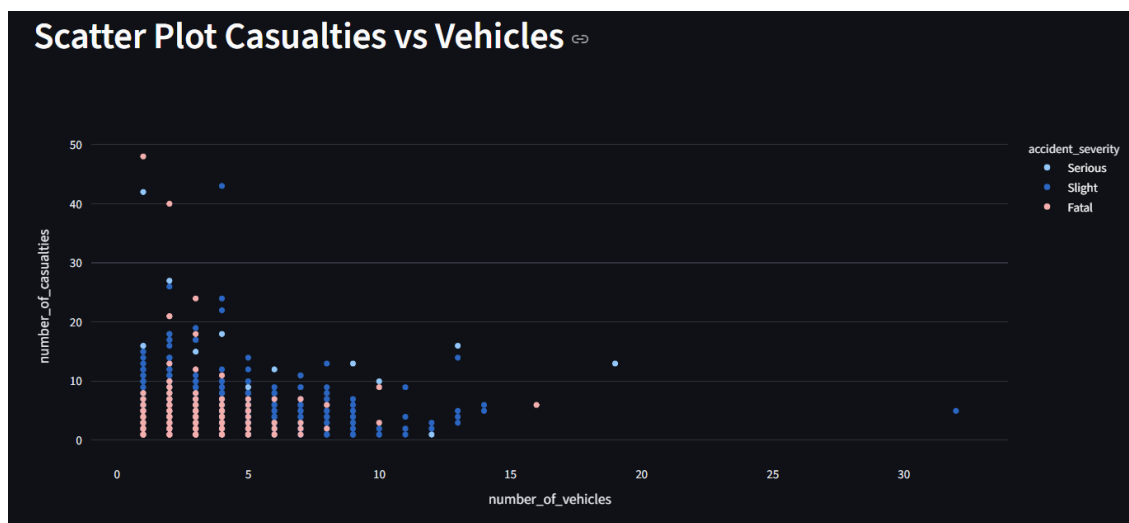


Gambar 27. Heatmap Hotspot Kecelakaan

5.9 Analisis Scatter Plot Casualties vs Vehicles

Scatter plot casualties vs vehicles menunjukkan hubungan antara jumlah kendaraan yang terlibat dan jumlah korban dalam kecelakaan. Sebagian besar data berada pada jumlah kendaraan yang rendah, yaitu sekitar satu sampai beberapa kendaraan, dengan jumlah korban yang juga relatif rendah.

Namun, terdapat beberapa outlier yang menunjukkan jumlah korban tinggi meskipun jumlah kendaraan tidak banyak. Hal ini menunjukkan bahwa banyaknya korban tidak selalu ditentukan oleh banyaknya kendaraan yang terlibat. Faktor lain seperti kecepatan, jenis kendaraan, kondisi jalan, lokasi kecelakaan, dan tingkat keparahan kecelakaan juga dapat memengaruhi jumlah korban.

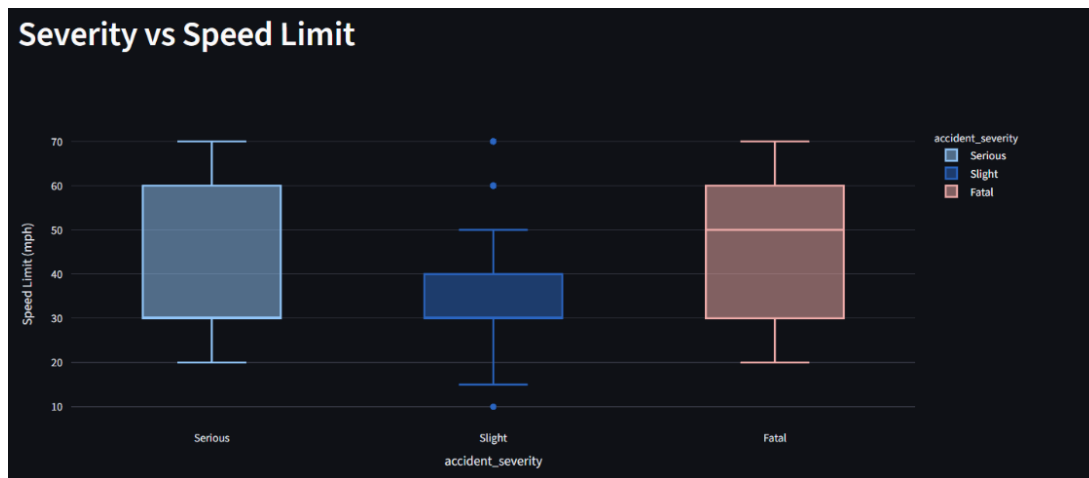


Gambar 28. Scatter Plot Casualties vs Vehicles

5.10 Analisis Saverity vs Speed Limit

Visualisasi severity vs speed limit menggunakan boxplot untuk membandingkan batas kecepatan pada setiap tingkat keparahan kecelakaan. Dari grafik, kategori Slight banyak berada pada speed limit yang lebih rendah, terutama sekitar 30 mph. Sementara itu, kategori Serious dan Fatal memiliki sebaran speed limit yang lebih lebar dan terlihat banyak berada pada rentang speed limit yang lebih tinggi.

Hasil ini menunjukkan bahwa lingkungan jalan dengan speed limit lebih tinggi dapat berkaitan dengan tingkat keparahan kecelakaan yang lebih besar. Kecepatan yang lebih tinggi dapat meningkatkan dampak saat terjadi tabrakan. Namun, hasil ini tetap perlu dibaca sebagai pola deskriptif, bukan hubungan sebab-akibat langsung. Untuk membuktikan pengaruh speed limit terhadap severity, diperlukan analisis statistik lanjutan.



Gambar 29. Saverity vs Speed Limit

5.11 Ringkasan Hasil Visualisasi Dashboard

Berdasarkan hasil visualisasi dashboard, dapat disimpulkan bahwa pola kecelakaan jalan raya di Inggris didominasi oleh kecelakaan dengan kategori Slight. Jenis kendaraan yang paling banyak terlibat adalah car, sedangkan jenis jalan dengan jumlah kecelakaan tertinggi adalah single carriageway. Dari sisi wilayah, Birmingham menjadi district dengan jumlah kecelakaan tertinggi, diikuti oleh beberapa district besar lainnya seperti Leeds, Manchester, dan Bradford.

Dari sisi waktu, kecelakaan cenderung meningkat pada jam sibuk, terutama pagi dan sore hari. Jumlah kecelakaan pada weekday juga lebih tinggi dibandingkan weekend, yang kemungkinan berkaitan dengan aktivitas harian dan mobilitas kerja. Dari sisi spasial, heatmap menunjukkan adanya konsentrasi kecelakaan pada beberapa area tertentu. Sementara itu, scatter plot menunjukkan bahwa jumlah korban tidak selalu berbanding lurus dengan jumlah kendaraan yang terlibat. Boxplot speed limit juga menunjukkan bahwa kecelakaan Fatal dan Serious memiliki sebaran speed limit yang lebih tinggi dibandingkan Slight.

Secara umum, dashboard yang dibuat sudah cukup membantu dalam membaca pola kecelakaan jalan raya di Inggris. Informasi yang ditampilkan tidak hanya menunjukkan jumlah kecelakaan, tetapi juga memperlihatkan pola berdasarkan tingkat keparahan, waktu kejadian, jenis kendaraan, jenis jalan, lokasi, jumlah korban, dan batas kecepatan. Dengan adanya visualisasi ini, data kecelakaan menjadi lebih mudah dipahami dan dapat digunakan sebagai bahan awal untuk melihat kondisi atau area yang perlu mendapat perhatian dalam keselamatan lalu lintas.

BAB VI

KESIMPULAN

6.1 Kesimpulan

Berdasarkan proyek yang telah dilakukan, sistem analisis kecelakaan jalan raya di Inggris berhasil dikembangkan menggunakan Apache Spark, PostgreSQL, MongoDB, Streamlit, dan Plotly. Apache Spark digunakan untuk mengolah data, mulai dari membaca dataset, membersihkan data, membuat fitur baru, hingga melakukan analisis. Data yang sudah diproses kemudian disimpan dan digunakan untuk membuat dashboard visualisasi.

Hasil analisis menunjukkan bahwa kecelakaan paling banyak berada pada kategori Slight. Jenis kendaraan yang paling sering terlibat adalah Car, sedangkan jenis jalan dengan jumlah kecelakaan tertinggi adalah Single carriageway. Dari sisi wilayah, Birmingham menjadi district dengan jumlah kecelakaan tertinggi, diikuti oleh Leeds, Manchester, dan Bradford.

Dari sisi waktu, kecelakaan lebih sering terjadi pada jam sibuk, terutama pagi dan sore hari. Jumlah kecelakaan pada weekday juga lebih tinggi dibandingkan weekend. Selain itu, heatmap menunjukkan adanya beberapa area dengan konsentrasi kecelakaan yang lebih tinggi. Scatter plot memperlihatkan bahwa jumlah korban tidak selalu bergantung pada jumlah kendaraan yang terlibat, sedangkan boxplot speed limit menunjukkan bahwa kecelakaan Fatal dan Serious cenderung memiliki sebaran batas kecepatan yang lebih tinggi.

Secara keseluruhan, dashboard yang dibuat sudah mampu menampilkan hasil analisis dengan jelas dan mudah dipahami. Dashboard ini membantu pengguna melihat pola kecelakaan berdasarkan tingkat keparahan, waktu, jenis kendaraan, jenis jalan, lokasi, jumlah korban, dan speed limit. Hasil dari proyek ini dapat digunakan sebagai gambaran awal untuk memahami pola kecelakaan dan melihat area yang perlu mendapat perhatian dalam keselamatan lalu lintas.

6.2 Saran

Dashboard juga dapat dikembangkan dengan menambahkan fitur ringkasan utama, seperti total kecelakaan, total korban, jumlah district, dan rata-rata kecelakaan berat. Selain itu, visualisasi clustering K-Means dapat ditampilkan secara khusus agar hasil hotspot kecelakaan lebih jelas dan tidak tercampur dengan heatmap biasa. Dengan pengembangan tersebut, dashboard dapat menjadi lebih lengkap dan lebih mudah digunakan untuk analisis keselamatan jalan.

Referensi

- [1] H. O. World, *Global status report on road safety 2023*. Geneva: World Health Organization, 2023. [Online]. Available: <https://www.who.int/publications/i/item/9789240086517>
- [2] Department For Transport, “Reported Road Casualties Great Britain: Annual Report 2023.” London, 2024. [Online]. Available: <https://www.gov.uk/government/statistics/reported-road-casualties-great-britain-annual-report-2023>
- [3] M. Younas, “Research challenges of big data,” *Serv. Oriented Comput. Appl.*, vol. 13, no. 2, pp. 105–107, 2019, doi: 10.1007/s11761-019-00265-x.
- [4] N. Orcid, “Big Data Characteristics (V ’ s) in Industry,” vol. 8, pp. 1–9, 2021, doi: 10.53523/ijoirVol8I1ID52.
- [5] S. Kliment, F. Information, and C. Technologies, “Big Data Analytics in Medicine and Healthcare Abstract :,” pp. 1–5, 2018, doi: 10.1515/jib-2017-0030.
- [6] M. Zaharia *et al.*, “Resilient Distributed Datasets : A Fault-Tolerant Abstraction for In-Memory Cluster Computing”.
- [7] X. Meng *et al.*, “MLlib : Machine Learning in Apache Spark,” vol. 17, pp. 1–7, 2016.
- [8] T. Postgresql and G. Development, “PostgreSQL 16 . 14 Documentation The PostgreSQL Global Development Group”.
- [9] C. A. Gy and D. R. Zmaranda, “A Comparative Study of MongoDB and Document-Based MySQL for Big Data Application Data Management,” 2022.
- [10] A. Karudin, D. Leni, D. Y. Sari, Y. Fernanda, and Y. P. Kusuma, “Development of Practical Data-Based Visualization Models Using the Streamlit Framework in Thermodynamics Learning,” vol. 14, no. 1, pp. 913–924, 2025, doi: 10.18421/TEM141.